MLA0303_RL_PROGRAM_15 — Program with Output

Python Source Code

```
"""
Experiment 15: PPO and TRPO for a humanoid robot to achieve stable
walking and balance. A simplified 1-D balance environment is used
(the robot's torso tilt angle, discretized into bins) so both
algorithms' core mechanics -- PPO's clipped surrogate objective and
TRPO's trust-region (KL-constrained) update -- can be demonstrated
compactly with a tabular softmax policy.
"""

import numpy as np

np.random.seed(3)

N_BINS = 7           # tilt angle bins: 0=fully left .. 6=fully right, 3=balanced
CENTER = 3
ACTIONS = ["LEAN_LEFT", "LEAN_RIGHT", "HOLD"]
GAMMA = 0.95
EPISODES = 1500


class BalanceEnv:
    def reset(self):
        self.tilt = np.random.choice([1, 2, 4, 5])
        self.steps = 0
        return self.tilt

    def step(self, action):
        if action == 0:
            self.tilt = max(0, self.tilt - 1)
        elif action == 1:
            self.tilt = min(N_BINS - 1, self.tilt + 1)
        self.steps += 1
        if self.tilt in (0, N_BINS - 1):
            return self.tilt, -20, True             # fell over
        reward = 5 if self.tilt == CENTER else -abs(self.tilt - CENTER)
        done = self.steps >= 25
        return self.tilt, reward, done


def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()


def collect_episode(theta, env):
    state = env.reset()
    traj = []
    for _ in range(30):
        probs = softmax(theta[state])
        action = np.random.choice(len(ACTIONS), p=probs)
        next_state, reward, done = env.step(action)
        traj.append((state, action, reward, probs.copy()))
        state = next_state
        if done:
            break
    return traj


def discounted_returns(traj):
    G, returns = 0, [0] * len(traj)
    for t in reversed(range(len(traj))):
        G = traj[t][2] + GAMMA * G
        returns[t] = G
    return returns


def train_ppo(episodes=EPISODES, clip_eps=0.2, lr=0.05):
    theta = np.zeros((N_BINS, len(ACTIONS)))
    env = BalanceEnv()
    baseline = 0.0
    rewards_hist = []
    for ep in range(episodes):
        traj = collect_episode(theta, env)
        returns = discounted_returns(traj)
        ep_return = sum(r for _, _, r, _ in traj)
        baseline = 0.95 * baseline + 0.05 * ep_return
        for (s, a, r, old_probs), G_t in zip(traj, returns):
            advantage = G_t - baseline
```

```
            new_probs = softmax(theta[s])
            ratio = new_probs[a] / (old_probs[a] + 1e-8)
            clipped_ratio = np.clip(ratio, 1 - clip_eps, 1 + clip_eps)
            surrogate = min(ratio * advantage, clipped_ratio * advantage)
            grad = -new_probs
            grad[a] += 1
            theta[s] += lr * surrogate * grad          # PPO clipped-surrogate update
        rewards_hist.append(ep_return)
    return theta, rewards_hist


def train_trpo(episodes=EPISODES, kl_limit=0.02, lr=0.05):
    theta = np.zeros((N_BINS, len(ACTIONS)))
    env = BalanceEnv()
    baseline = 0.0
    rewards_hist = []
    for ep in range(episodes):
        traj = collect_episode(theta, env)
        returns = discounted_returns(traj)
        ep_return = sum(r for _, _, r, _ in traj)
        baseline = 0.95 * baseline + 0.05 * ep_return
        for (s, a, r, old_probs), G_t in zip(traj, returns):
            advantage = G_t - baseline
            proposed_theta = theta[s].copy()
            grad = -old_probs
            grad[a] += 1
            proposed_theta += lr * advantage * grad
            new_probs = softmax(proposed_theta)
            kl = np.sum(old_probs * np.log((old_probs + 1e-8) / (new_probs + 1e-8)))
            # Trust-region check: scale the step back if the KL divergence
            # between old and new policy exceeds the allowed limit.
            scale = 1.0 if kl <= kl_limit else np.sqrt(kl_limit / (kl + 1e-8))
            theta[s] += scale * lr * advantage * grad
        rewards_hist.append(ep_return)
    return theta, rewards_hist


if __name__ == "__main__":
    theta_ppo, rewards_ppo = train_ppo()
    theta_trpo, rewards_trpo = train_trpo()

    print(f"{'Algorithm':<10}{'Avg reward (first 100 ep)':<28}{'Avg reward (last 100 ep)'}")
    print(f"{'PPO':<10}{np.mean(rewards_ppo[:100]):<28.2f}{np.mean(rewards_ppo[-100:]):.2f}")
    print(f"{'TRPO':<10}{np.mean(rewards_trpo[:100]):<28.2f}{np.mean(rewards_trpo[-100:]):.2f}")

    env = BalanceEnv()
    for name, theta in [("PPO", theta_ppo), ("TRPO", theta_trpo)]:
        env.tilt, env.steps = 1, 0
        path = [env.tilt]
        for _ in range(10):
            action = int(np.argmax(theta[env.tilt]))
            _, _, done = env.step(action)
            path.append(env.tilt)
            if done:
                break
        print(f"{name} balance recovery path from tilt=1: {path}")
```

Execution Output

```
Algorithm Avg reward (first 100 ep)   Avg reward (last 100 ep)
PPO       94.52                       122.42
TRPO      88.91                       105.18
PPO balance recovery path from tilt=1: [1, 2, 3, 3, 3, 3, 3, 3, 3, 3]
TRPO balance recovery path from tilt=1: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

STDERR:
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 772, in warm_spreadsheet_
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/connection.py", line 37, in get_or_create_client
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/daemon.py", line 124, in start_daemon
TimeoutError: Timed out waiting for artifact tool daemon socket. Set ARTIFACT_TOOL_RPC_DAEMON_STARTUP_TIMEOUT_S=<seconds
```